Modular Programming – Practical Work

Week 6

Inheritance

Notations: W1Q1 = Week 1, Questions of type 1

Question types:

- Type 1: theory, concepts — *knowledge*
- Type 2: understanding typically based on code reading — *understanding*
- Type 3: producing yourself code — *know-how*

A moodle deposit space is open every week for you to upload your solutions. Solutions of the exercises are provided one week later.

Important points:

- 602_F and 602_D classes should upload solutions **before Monday 24h00 the following week**
- **2 or 3 practical work will be corrected** and will count for one quarter of the intermediate test note

Conventions about fonts:

- `Lucida Console` is used for Java reserved commands
- `Courrier` is used for the code
- Arial is used for the text

W6Q3 – 1 Abstract Class

Write the classes (`TrafficSignal, TrafficLight, WalkSign`) for the traffic signals in the US Streets. Let's create the traffic lights and Walk/Don't Walk signs with the notion of abstract class. Here are some of the specifications for the project:

1. Traffic lights have an intermediate yellow state, and always pass through their four possible states in the cyclic order

   green→yellow→red→yellow→green…

   

2. Walk/Don't Walk signs have an intermediate flashing don't walk state, and always pass through their four possible states in the cyclic order :

   walk→flashing don't walk→don't walk→flashing don't walk→walk…

   

3. By default, the traffic light is green and the pedestrian must wait (don't walk). To simplify, when the traffic light becomes yellow, the message for the pedestrian is "flashing don't walk" and when the traffic light is red, the message is "walk".

4. Create a loop within a main method, that changes states (all the states once) of the two traffic signals and displays every time the result.

   Results on screen :

```
[class WalkSign:  don't walk]
[class TrafficLight:  green]
**********************
[class WalkSign:  flashing don't walk]
[class TrafficLight:  yellow]
**********************
[class WalkSign:  walk]
[class TrafficLight:  red]
**********************
[class WalkSign:  flashing don't walk]
[class TrafficLight:  yellow]
**********************
[class WalkSign:  don't walk]
[class TrafficLight:  green]
**********************
```

W6Q3 – 2 Inheritance

Create 7 classes (generate the needed getters/setters), regarding the following rules :

1. Message
   - Properties private String text
   - Methods public void send() // Display the text in the console (because we don't know yet how to send a mail, but the name of the method would be wrong to only display !)

2. Recipient
   - Properties private String name, firstName
   - Constructors public Recipient(String name, String firstName)

3. MailRecipient // extends from Recipient class
   - Properties private String mailAddress
   - Constructors public MailRecipient(String name, String firstName, String mailAddress)

4. SMSRecipient // extends from Recipient class
   - Properties private String phoneNumber
   - Constructors public SMSRecipient(String name, String firstName, String phoneNumber)

5. MailMessage // extends from Message class
   - Properties private static final NBCHARMAX = 600 // Could be more
     MailRecipient recipient // If only one recipient
     MailRecipient[] recipients // If several recipients
   - Constructors // You can choose between these two constructors, using the class. Whether you use one, or you use several recipients
     public MailMessage(String text, MailRecipient recipient)
     public MailMessage(String text, MailRecipient[] recipients)
   - Methodes private void sendToRecipient(MailRecipient recipient) // Hidden method that display the recipient the message is sent to
     public void send() // Display the full message, including all the recipients and the message. If only one recipient is defined

6. SMSMessage // extends from Message class
   - // Everything is the same as MailMessage, except it contains SMSRecipients

7. TestsMessage
   // Tests the two classes MailMessage and SMSMessage and their send() methods. Use the other classes if needed.